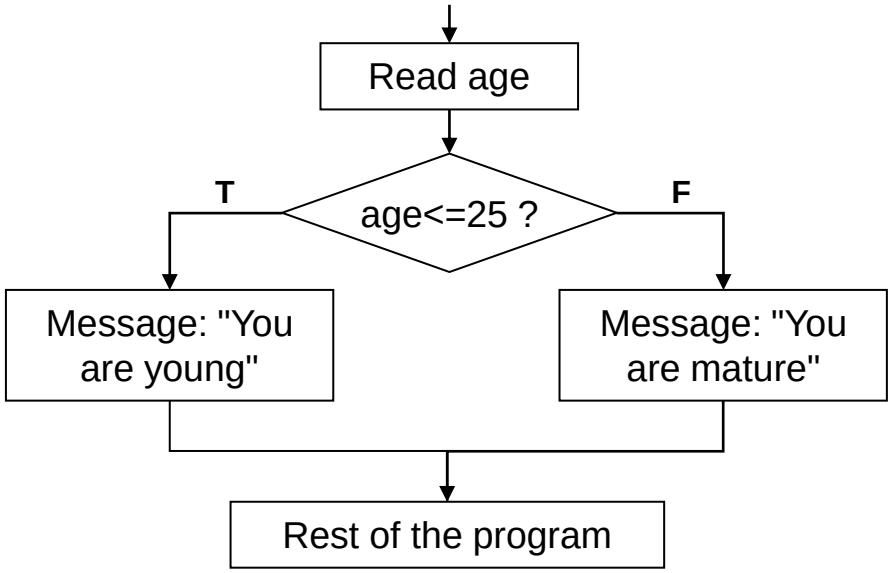
If statement

- The "if statement" is used to break the sequential flow of execution.
- Enforces branching in execution according to the result of an expression.
  - There are two possible paths to take.
  - The expression determines which path should be taken.



If statement

- Syntax:

```
if (int_expr)
    stat_block1
[else
    stat_block2 ]
```

Text in green is optional

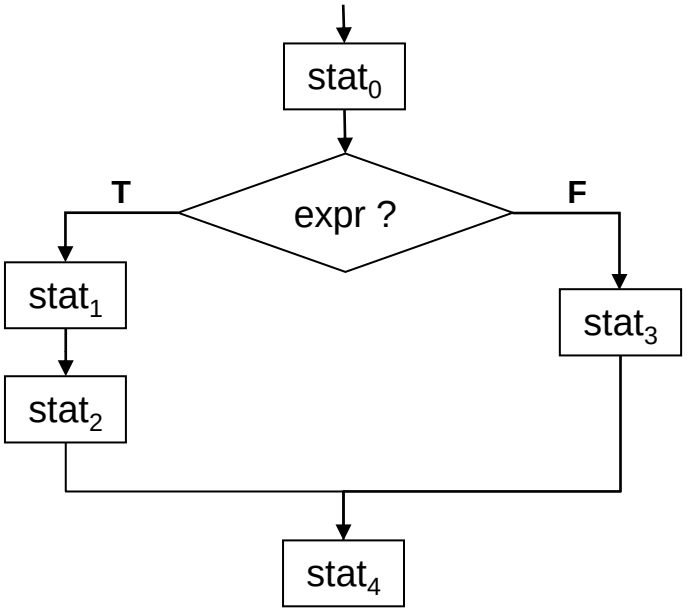
where **stat_block** is one of the following:

- a single statement `stat;`
- the null statement `;`
- a group of statements enclosed in braces

```
{   stat1;
    ...
    statn;
}
```

If statement

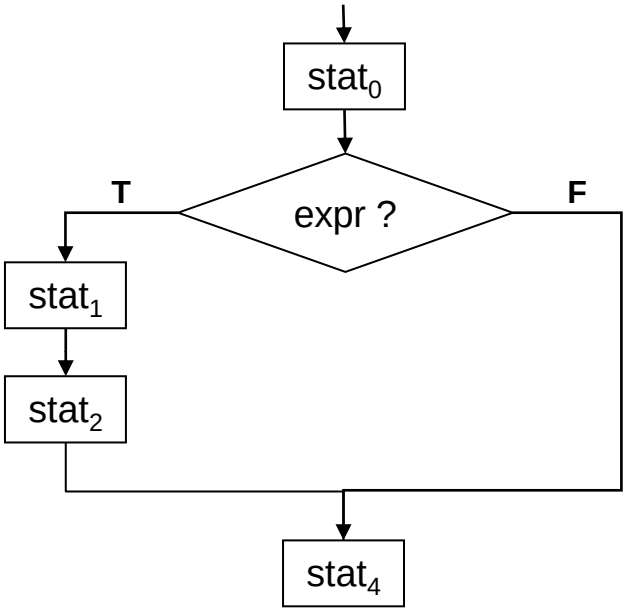
```
stat0;
if (expr)
{   stat1;
    stat2;
}
else
    stat3;
stat4;
```



Notice the indentation.

If statement

```
stat0;
if (expr)
{   stat1;
    stat2;
}
stat4;
```



How would you move $stat_1$ and $stat_2$ to the "then" branch?

If statement

- Read one character as input; check if it is a digit; if so, convert it to an integer and display twice that number; o/w display an error message.

```
char ch;   int num;

scanf("%c", &ch);
if (('0'<=ch) && (ch<='9'))
{  num=ch-'0';
   printf("Twice input is %d \n", 2*num);
}
else
   printf("Input is not a digit! \n");
```

If statement

- Read a number and state whether it is odd or even.

```
int num;

scanf("%d", &num);
printf("%d is an ", num);
if (num%2!=0)
   printf("odd ");
else
   printf("even ");
printf("number.\n");
```

Nested-if statements

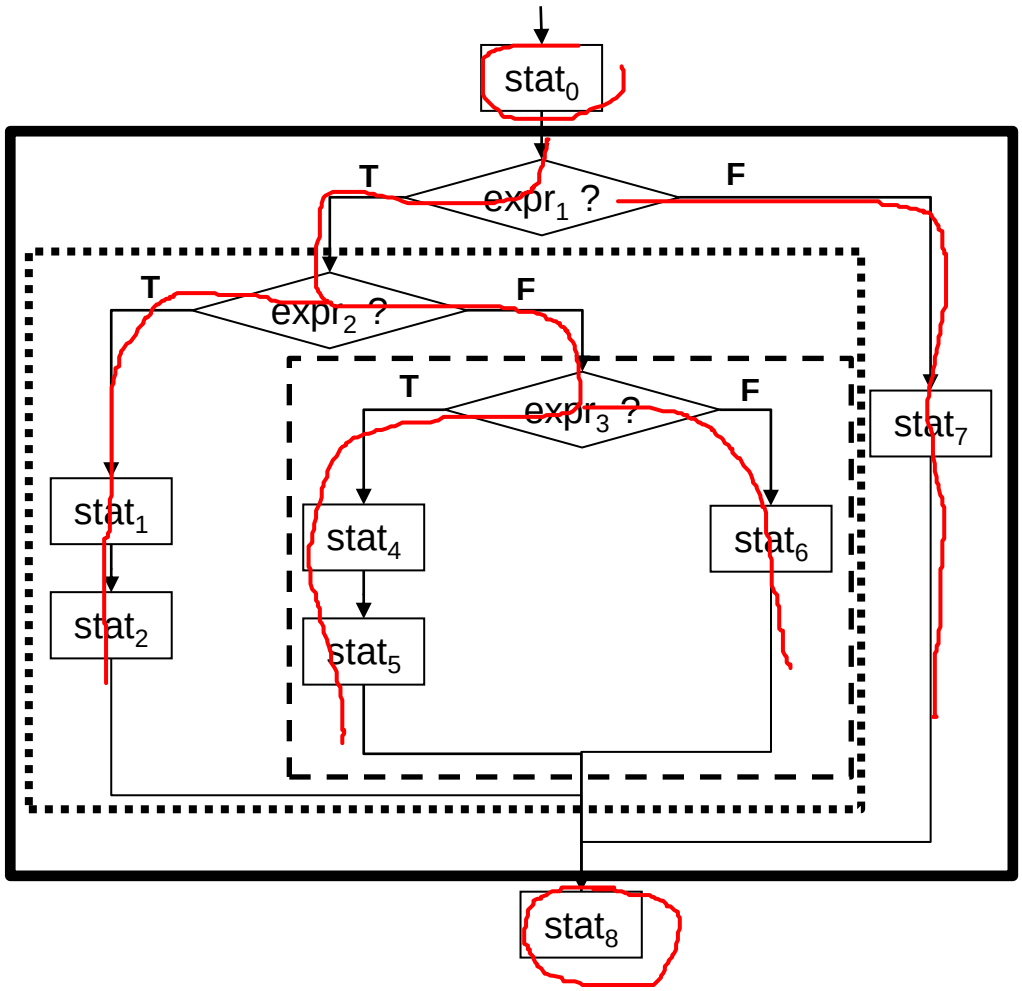
- Remember the syntax:

```
if (int_expr)
    stat_block1
else
    stat_block2
```

- Statement block contains statements.
  - "if" is also a statement.
  - So, you can "nest" one if statement in another.
    - This structure is called nested-if statements.
    - You can nest as much as you can.

Nested-if statements

```
stat0;
if (expr1)
   if (expr2)
   {  stat1;
      stat2;
   }
   else
      if (expr3)
      {  stat4;
         stat5;
      }
      else
         stat6;
else
   stat7;
stat8;
```

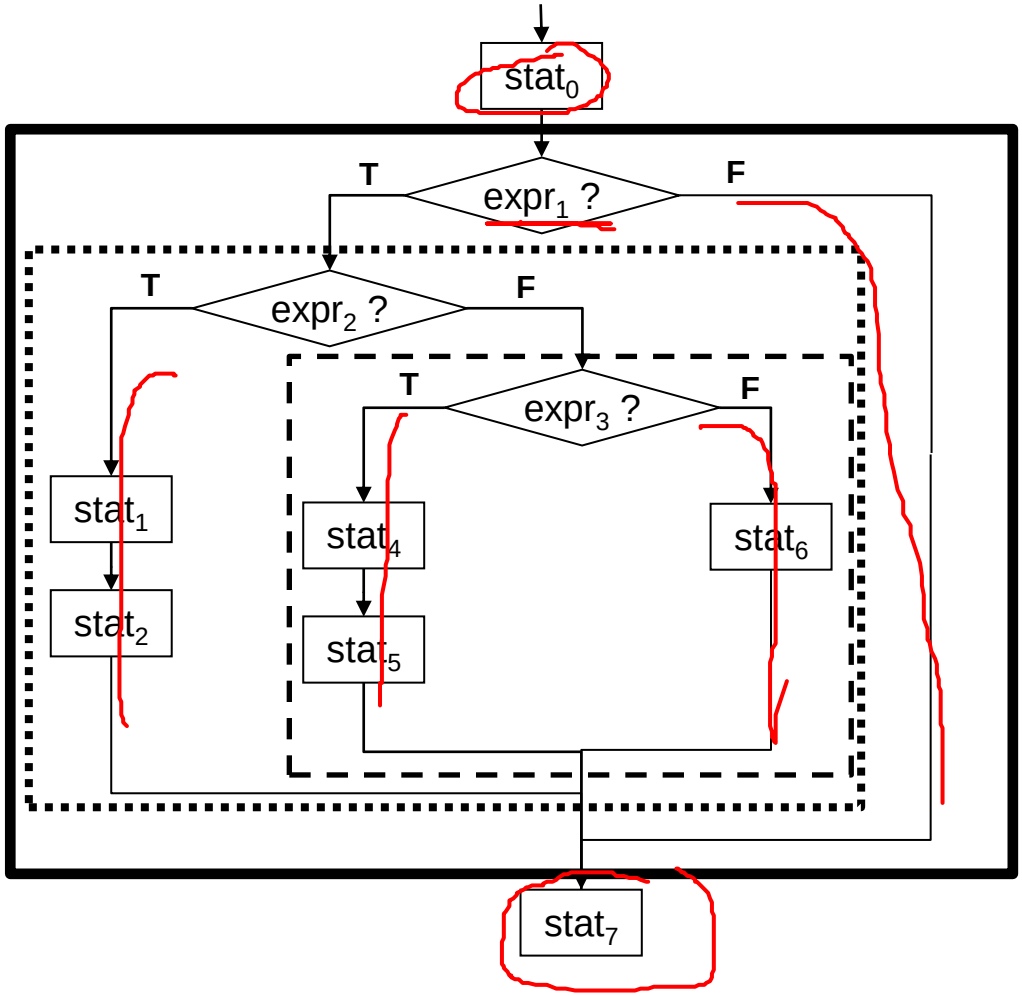


Nested-if statements

- Remember that the "else" part is optional.
  - Thus, some of the if statements may not have an "else" part.
  - Then, it is a little bit tricky to find to which "if" the "else" belongs.
    - Note that indentation is completely ignored by the compiler, so it does not help.
- The trick is the following:
  - "else" belongs to the nearest incomplete "if"

Nested-if statements

```
stat0;
if (expr1)
   if (expr2)
   {  stat1;
      stat2;
   }
   else
      if (expr3)
      {  stat4;
         stat5;
      }
      else
         stat6;
stat7;
```



Else-if statements

- Else-if is a variation of nested-if.
- An inner if statement is executed iff all previous if statements have failed.
  - Thus, executing an inner if statement *implies* that all previous expressions were false.
- Syntax:

```
if (int_expr1)
    stat_block1
else if (int_expr2)
    stat_block2
...
else
    stat_blockn
```

Else-if statements

```
if (age<=1)
   printf("infant");
else if (age<=3)
   printf("toddler");
else if (age<=10)
   printf("child");
else if (age<=18)
   printf("adolescent");
else if (age<=25)
   printf("young");
else if (age<=39)
   printf("adult");
else if (age<=65)
   printf("middle-aged");
else
   printf("elderly");
```

Else-if statements

- Alternative would be:

```
if (age<=1)
   printf("infant");
if ((1<age) && (age<=3))
   printf("toddler");
if ((3<age) && (age<=10))
   printf("child");
if ((10<age) && (age<=18))
   printf("adolescent");
if ((18<age) && (age<=25))
   printf("young");
if ((25<age) && (age<=39))
   printf("adult");
if ((39<age) && (age<=65))
   printf("middle-aged");
if (65<age)
   printf("elderly");
```

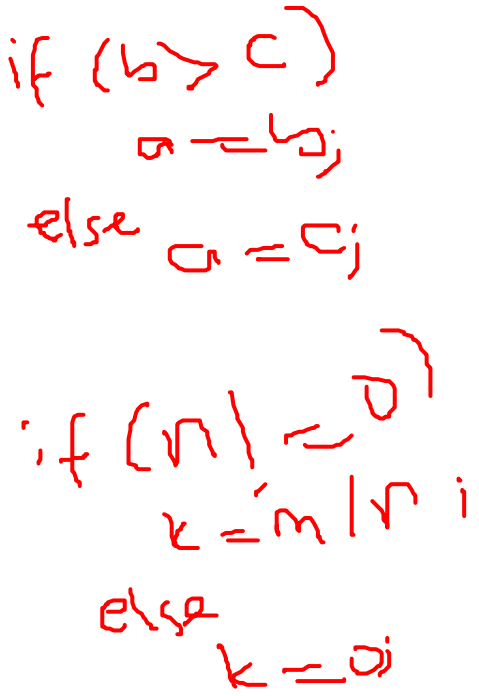
Ternary operator

- Ternary operator is similar to the "if" statement. But it is an operator, not a statement.
- Syntax:

```
int_expr ? value1 : value2
```

- Eg:

```
a = (b>c) ? b : c;
k = (n!=0) ? m/n : 0;
```



Ternary operator

- Note that it is not possible to know at compile time whether **`value`**$_1$ or **`value`**$_2$ will be used.
  - Therefore, the type of the expression is the type of the larger value.
- Eg: In the expression below, if the value of **`b`** is **`9`**

```
a = b / (b%2)?2:3.0;
```

the value of **`a`** is **`4.5`** (not **`4`**), because we perform a float division (not integer division)